



Defect-scanner: a comparative empirical study on language model and deep learning approach for software vulnerability detection

Van-Hau Pham^{1,2} · Do Thi Thu Hien^{1,2} · Hien Do Hoang^{1,2} · Phan The Duy^{1,2}

Published online: 13 August 2024

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

The complex and rapidly evolving nature of modern software landscapes introduces challenges such as increasingly sophisticated cyber threats, the diversity in programming languages and coding styles, and the need to identify subtle patterns indicative of vulnerabilities. These hurdles underscore the necessity for advanced techniques that can effectively cope with the intricacies of software security. Hence, this paper gives a comparative empirical study in harnessing the potential of cutting-edge natural language processing (NLP) advancements, namely Word2Vec and CodeBERT to detect vulnerabilities in C and C++ programs in the proposed Defect-Scanner framework. With the capability of converting code components and source code into contextual embedding vectors, various potential NLP techniques are combined with several DL models to evaluate the precision and accuracy of identifying vulnerabilities within software systems. Moreover, the experimentations are conducted using datasets with different representation types of codes, aiming to figure out the best combination of NLP techniques and DL models to work with each form of input. As a result, besides the outperformance of CodeBERT-based models with accuracies of approximately 90%, this comparative study also provides a comprehensive evaluation of NLP-based software vulnerability detection in the face of intricate security challenges.

Keywords Software vulnerability detection · Deep Learning · Natural Language Processing · Language Model · CodeBERT.

1 Introduction

In the dynamic landscape of software development, the assurance of application security and robustness holds paramount importance. The proliferation of intricate software systems, coupled with the ever-evolving array of attack vectors, underscores the critical need for effective software vulnerability detection techniques [6, 10].

Traditional vulnerability detection approaches, exemplified by rule-based scanners, have demonstrated their util-

ity in identifying known patterns of vulnerabilities within codebases. Despite their proven usefulness, the possible limitations in comprehensively identifying vulnerabilities underscore the necessity for innovative strategies that leverage advancements in Machine Learning (ML) and Deep Learning (DL) to fortify software security [40, 43]. Moreover, when it comes to analyzing source codes, the existing common methods often grapple with the complexities of modern software systems, struggling to capture the nuanced context embedded within source code representations. As a result, they may miss intricate vulnerabilities that demand a more profound understanding of code semantics and interrelationships [11, 12, 20].

In the realm of software vulnerability detection, the integration of ML and DL models enhances the precision and scalability of the detection process [7]. By training on diverse datasets containing both vulnerable and non-vulnerable code, sophisticated algorithms can be developed to discern subtle indicators of vulnerabilities, even within complex codebases [14, 23]. In such a case, where textual interpreting to comprehend semantics and contexts plays a vital role, Natural Language Processing (NLP) can be a promising solution.

✉ Van-Hau Pham
haupv@uit.edu.vn

Do Thi Thu Hien
hiendtt@uit.edu.vn

Hien Do Hoang
hiendh@uit.edu.vn

Phan The Duy
duypt@uit.edu.vn

¹ Information Security Laboratory, University of Information Technology, Ho Chi Minh City, Vietnam

² Vietnam National University, Ho Chi Minh City, Vietnam

Being renowned for its proficiency in comprehending human languages and intricate contexts [15], NLP has exhibited its potential across diverse domains, including software security. When applied to the analysis of source code, NLP techniques can decode complex relationships within code components [36]. This has the potential to lead to the identification of vulnerabilities that might evade rule-based scanners, thereby facilitating a more comprehensive assessment of a software system's security. Furthermore, the advancements in ML and DL offer unprecedented opportunities to develop models capable of generalizing from extensive datasets, enabling the identification of intricate patterns and anomalies as in the existing works [18, 19, 45]. While Word2Vec [25] is the most well-known word embedding technique taken advantage of in those NLP-based works to equip them with the ability to comprehend semantics in software's source code, there are still other word-embedding methods, such as CodeBERT [9], with the promising potential that needs to be explored in the field of software security.

Motivated by the mentioned promising solutions, this paper embarks on a comprehensive exploration of an approach that harnesses the potential of NLP and deep language models, with the advancements in ML and DL. By bridging the gap between intricate source code nuances and effective vulnerability detection, this approach demonstrates the transformative potential of these advancements. Through rigorous experimental validation and comprehensive comparative analysis, the advantages, and effectiveness of this approach are demonstrated in stark contrast to the limitations of traditional methods of vulnerability detection. This approach stands at the intersection of NLP, ML, and software security, marking the advent of a new era in software vulnerability detection. By capitalizing on the rich contextual nature of source code and leveraging the strides made in ML/DL, this approach holds the promise of elevating the standards of vulnerability identification and augmenting software resilience against the ever-evolving spectrum of security threats.

To sum up, we outline the primary contributions of this study as follows.

- Design and employ an NLP-assisted model for software vulnerability detection, where source code can be represented in various forms and then transformed using different NLP-assisted embedding approaches to have suitable input to feed into various DL models.
- Conduct experiments to evaluate the effectiveness of different combinations of word embedding techniques and DL models on the overall vulnerability detection performance in dealing with different representing forms of source code in various datasets.

The remaining sections of this article are constructed as follows. Section 2 reviews some background knowledge relating to software vulnerability detection, followed by reviewing some related works in applying NLP in the field of software security. Next, the details of the proposed NLP-based system, called Defect-Scanner, to effectively detect vulnerability in software are discussed in Sect. 3. Section 4 describes the experimental settings and scenarios to evaluate the performance of the proposed solutions. After Sect. 5 discussing the result as well as limitations and possible future work, we conclude the paper in Sect. 6.

2 Background and related works

This section provides some backgrounds and terminologies in the field of software vulnerability detection. Besides, a review of the application of ML/DL and NLP techniques in addressing this problem is also mentioned.

2.1 Threat to Validity

Like other studies on applying DL in specific cybersecurity domains, the relevant papers for conducting a literature review and comparative empirical study depend on the keyword and database searched [30, 31]. While we adhered to a comprehensive and systematic methodology to review the literature on DL and NLP-based approaches to code vulnerability detection, focusing specifically on studies published from 2018 to 2023, there are inherent limitations regarding the search and collection of relevant research. The primary threat to the validity of our survey is the reliance on keyword-driven searches across academic platforms such as Google Scholar, ACM Digital Library, IEEE Xplore, ScienceDirect, and SpringerLink.

Given the rapid evolution of terminology and methods in the fields of DL and NLP since 2018, there is a risk that pertinent studies might be overlooked if they employ emerging keywords or concepts not included in our initial search parameters. For example, recent terms like “ode embedding for vulnerability detection” or “L-based software vulnerability” may not have been fully captured if they were not part of our predefined search keywords. Moreover, our dependence on specific keywords such as “deep learning-based code vulnerability detection”, “language model for software vulnerability detection” and “LP-based code vulnerability detection” might restrict the scope of our collected literature to studies that explicitly use these terms, possibly excluding relevant research that describes similar concepts under different terminologies or newer methodologies. Hence, we use all the above-mentioned queries to search and filter the relevant papers in the domain of NLP-based software vulnerability detection. Additionally, selected papers are also filtered if

there is a published code repository from the authors of the papers. Please note that despite employing distinct search queries, there remains a possibility that some relevant papers were overlooked because authors sometimes use unique terminology that is not widely recognized.

2.2 Software Vulnerability

Software vulnerability refers to weaknesses, misconfigurations, or errors in various phases of the software development process, including designing, implementing, and operating the software. Those vulnerabilities can lead to the software being attacked or intruded and security policy violations.

Software vulnerabilities can be categorized into various groups based on their severity or occurrence within specific phases of the software development process.

- *Design Flaws* appear in the design phase, where the software or system architecture is outlined without or with partial security consideration.
- In the implementation phase, *programming flaws* can be caused when coding without security policy compliance, such as insufficient memory usage, or utilizing insecure/deprecated programming libraries. As a result, *programming vulnerabilities* can then exist under a specific condition, which makes the software vulnerable to attacks.
- *Operating System Vulnerabilities* relate to operations at the OS level such as I/O process, network activities, or access control.

Regardless of their positions or scopes, those vulnerabilities can be taken advantage of by attackers with sufficient knowledge and skills to gain unauthorized access to the system, modify, remove, or steal valuable data, perform harmful actions on the system, or other types of attacks.

2.3 Software Vulnerability detector

Due to the advancement of technologies that can allow attackers to perform more complicated attacks, there is an emerging need for effective solutions to detect vulnerabilities existing in pieces of software. This task can be performed either statically or dynamically. In the former approach, the detection is based on analyzing the source code manually or using supporting tools without executing that software. Meanwhile, the latter approach aims to observe the behaviors of software during running time in a dedicated and controlled environment to figure out abnormal activities. Moreover, some hybrid approaches are also considered by combining all the advantages of static and dynamic analysis.

2.4 DL and NLP in software vulnerability detection

With the advancement and promising results of Artificial Intelligence (AI) in solving various research problems, many efforts have been made to apply DL in the field of software security to detect vulnerabilities.

To leverage the outstanding capability of ML/DL models, several works paid their effort in emerging an effective representation of programs to use as input of models for vulnerability detecting. For example, using graph-based representation, such as Abstract Syntax Tree (AST), control flow graphs (CFG), or Control data-flow graphs (CDGF), to understand the indeed structural characteristics of programs is a promising approach in recent publications [17, 34, 35, 42]. Take, for example, the work of Wang et al. [35] which used a Deep Belief Network (DBN) model to directly learn semantic features from the source code using vectors extracted from AST to predict the possible vulnerabilities automatically. Using the same idea based on AST, another work by Li et al. [17], called DP-CNN, leveraged a Convolutional Neural Network (CNN) to generate corresponding features and then combined them with other traditional ones to represent programs in the task of identifying vulnerabilities in source codes. Besides, research by Viet Phan et al. [34] focused on extracting Control Flow Graphs (CFGs) from assembly codes to represent semantics more effectively and precisely, even with some tiny differences that traditional features or ASTs could not offer distinguishable results. Moreover, their work applied a graph-based CNN model to automatically learn the useful features in graph representation. Especially, the rising potential of Graph Neural Networks, which are designed to deal with graph-structured data, provides a better means to understand graph nature, motivating various research using graph-based features of software programs [5, 32].

Meanwhile, Li et al. [19] introduced a representation form of source code called code gadget, which can be used as the input for DL models. Later, the authors developed their previous work to an extended version [18], which proposed a framework to capture syntax and semantic information from source code in the form of SeVCs to enhance the performance of DL-based vulnerability detectors. The effectiveness of the suggested representation form has been proved in detecting vulnerabilities with the BGRU model.

Especially, due to the textual nature of source code, natural language processing (NLP) techniques are also potential in analyzing software to detect vulnerabilities. For example, Wang et al. [35] and Li et al. [17] both utilized simple word embedding techniques to encode tokens from ASTs into suitable numeric vectors, which are used as input for DBN or CNN model to extract the semantics of source code for later classification. Meanwhile, VulDeePeckper [19] and VulBERT [16] took advantage of Word2Vec to embed

Table 1 Literature review of DL/ML-based software vulnerability detection

Work	Extracted information	Embedding method	DL/ML model	Dataset	Target language
Wang et al. [35]	Abstract syntax tree (AST)	Token-integer mapping and DBN-based feature generation	ADTree, Naïve Bayes (NB), logistic regression (LR)	PROMISE	Java
Li et al. [17]	Abstract syntax tree (AST), Traditional features	Token-integer mapping and CNN-based feature generation	LR	PROMISE	Java
Viet Phan et al. [34]	Control flow graph (CFG)	–	Directed Graph-based CNN (DGCNN)	CodeChef	C/C++
VulDeePecker [19]	Code gadget	Word2Vec	BiLSTM	NVD, SARD	C/C++
SySeVR [18]	SeVC	Word2Vec	BiLSTM, LSTM, GRU, BGRU, LR, MLP, CNN	NVD, SARD	C/C++
Noah et al. [44]	Source code	Tokenizing	LSTM, BiLSTM, BERT	SARD	C/C++
LineVD [13]	Function code and statements, program dependency graph (PDG)	CodeBERT and graph attention network (GAT)	MLP	SVD	C/C++
CSGVD [32]	Statements and control flow graph (CFG)	CodeBERT-based PE-BL embedding module and GCN	MLP	CodeXGLUE	C/C++
Defect-Scanner (Ours)	Code gadget, SeVC	Word2Vec, CodeBERT	BiLSTM, RNN, CNN, transformer	VulDeePecker, SySeVR	C/C++

tokens in code gadgets and SeVCs into vectors for training DL models. Besides, a work of Noah [44] treated source code as texts with words and sub-words and directly applied various NLP models to detect vulnerability. The authors evaluated various NLP models, including LSTM, BiLSTM, and BERT as well as their combinations, which indicated the effectiveness of the bidirectional nature in some models and the cooperation of BERT and other models in improving the accuracy of vulnerability detection. In another work of Hin et al. [13], CodeBERT—a rising pre-trained model for embedding source code is used to produce embeddings for overall function as well as each statement, respectively. As a result, this can help the proposed approach to detect function-level vulnerabilities and locate the vulnerable statements in code. Meanwhile, the authors of CSGVD [32] also consider the source code of a program as sequences and design their embedding module called PE-BL for node embedding to integrate these sequences in a CFG.

The summarization of related works is mentioned in Table 1. Although NLP techniques have their effectiveness for embedding tasks proven in some publications, most of these works only leverage a specific technique without considering other available ones for a comprehensive evaluation of various NLP-related approaches. Hence, this paper aims to consider multiple NLP techniques when accompanying various DL models in detecting vulnerability to verify the possible effect of choosing a suitable NLP technique for a DL model. Moreover, different representation forms of code are considered when preparing the input of NLP techniques

to figure out the most appropriate data form that should be used in NLP-assisted software vulnerability models.

3 Method

3.1 Terminology

In the scope of the proposed Defect-Scanner, the following terminologies are clarified for further understanding of its workflow.

3.1.1 Code representation

To offer a comprehensive understanding of a program's function, its codes can be split into smaller pieces, called code slices. A code slice refers to a part of the program's source code, consisting of multiple code lines related to a specific function or code block. This terminology is used to support the code-analyzing and vulnerability-finding process by creating smaller pieces of code to verify the corresponding security.

In the scope of this paper, the source code of a program can be represented in two following forms for later analysis tasks.

Code gadgets. A code gadget is made up of multiple code slices that are related in some aspects, such as data or control dependency. For instance, code gadgets proposed by Li et al. [19] are one of the means to represent the source

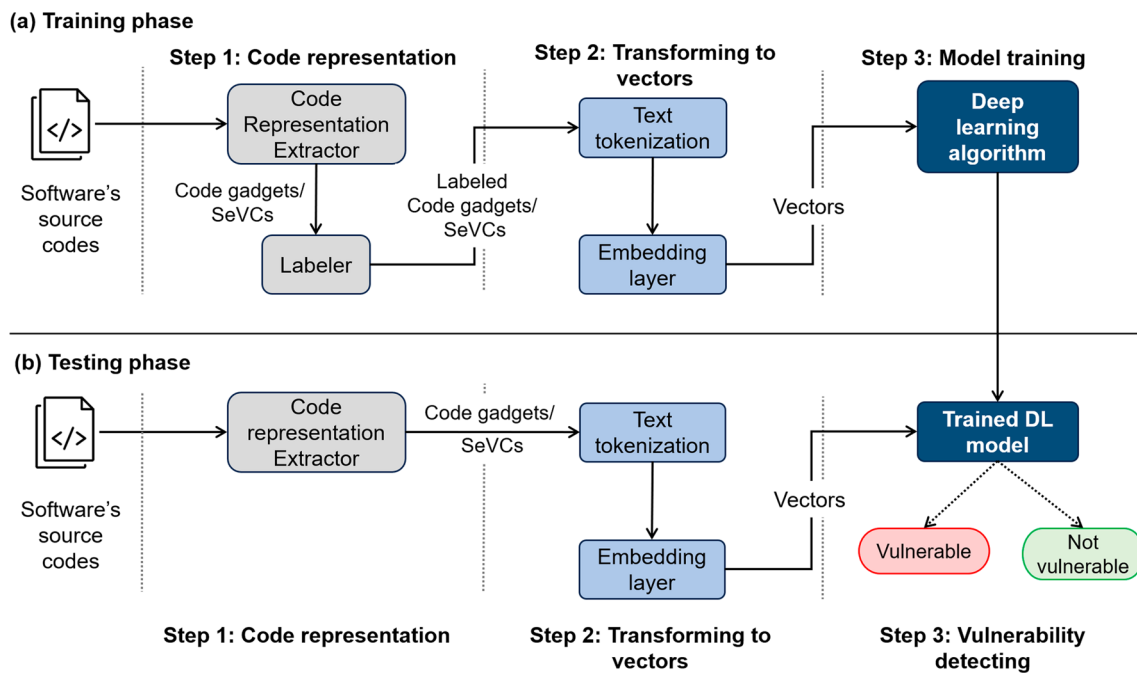


Fig. 1 Two phases of Defect-Scanner

code of the program, which are extracted via analyzing and tracking the semantic relation of code lines to an argument of a library or API calls. Besides, another version of code gadgets introduced by [16] focuses on statements relating to system calls as well as dealing with nested function calls. *Semantics-based Vulnerability Candidates (SeVCs)*. In addition to code gadgets, SeVCs introduced in a work of Li [18] are also used in vulnerability detection in Defect-Scanner. This type of code representation is extracted by identifying possible vulnerability candidates in source code that match any pre-defined vulnerability syntax characteristics, and then finding their related code lines to form the final SeVCs. This terminology reflects vulnerability syntax characteristics accommodated by semantic-related information.

Indeed, these representation forms are still a part of the program with multiple lines of code but only contain useful ones, which can be different according to the extracting approach, for recognizing vulnerability. Regardless of their forms, these code gadgets and SeVCs can be used to represent programs and then vectorized as input in deep learning model training.

3.1.2 Vulnerable software

A sample software is considered “vulnerable” if there is at least one vulnerability found in its source code. Otherwise, it is benign.

3.2 Overview of defect-scanner

As depicted in Fig. 1, the workflow of Defect-Scanner consists of 2 main phases of training and testing. The input of the training phase is numerous vulnerable as well as benign pieces of software, while the output of this process is the deep-learning model trained to recognize vulnerability patterns. Once the training has finished, the resulting model is used to verify other programs.

3.2.1 Training phase

With the source codes of programs as input, the training phase gets through multiple steps as in Fig. 1a.

Step 1: Code representation

In this first step, the source code is analyzed to extract its corresponding representative information in the form of code gadgets or SeVCs. This task is automatically performed by inheriting the proposed algorithm from [18, 19].

Moreover, labeling also takes place in this step of the training phase to assign code representations to their corresponding labels. This can be performed manually or by automatic tools. Note that, the scope of this paper focuses on a binary classification task for vulnerability detection, where the final result is one of two labels - vulnerable (1) or not vulnerable (0).

Step 2: Transforming to vectors using word embedding

To be used in training models, data needs to be in a suitable form, which is numerical in this case. Word embedding

methods are taken advantage of to result in vectors based on the extracted code gadgets or SeVCs. This task aims to result in numerical vectors that fully represent code gadgets or SeVCs and can be used for training deep learning models.

For more details, first, non-ASCII characters and comments in the code lines of code gadgets or SeVCs are removed as they do not play any role in vulnerability detection. Thereafter, user-defined functions or variables are renamed to unified forms such as FUNC1, FUNC2, or VAR1, VAR2 instead of using specific names. Then, each keyword, identifier, operator, or symbol in the code lines is considered a token in the text tokenization task. Later, those tokens are then mapped to corresponding represented vectors of numbers.

To have the numerical forms for tokens, different word embedding models can be applied as an embedding layer to produce fixed-length numerical vectors. For instance, this paper focuses on using Word2Vec [25] (particularly Skip-gram) and CodeBERT [9]. While Word2Vec generates vectors by mapping each token in code gadgets or SeVCs based on its occurrence context in the dataset, CodeBERT uses a neural network to create numerical vectors based on the ability to code semantics understanding and modeling. Using the tokenized code gadgets or SeVCs as the input, those embedding models map each token with a corresponding vector, and the set of all those vectors makes up the full representation for the given code gadget or SeVC.

However, due to differences in the number of tokens in code gadgets and SeVCs, their embedding may have various sizes while most deep-learning models require equal-length vectors as their inputs. Hence, there is an additional task to ensure the same-size embedding result via a fixed number of T tokens being processed. For more details, a 0-padding or cutting task is performed on the lists of tokens to achieve the same number of T tokens. Note that, this T value is customizable.

The output of this transforming step for each inputted code gadget or SeVC is a set of T vectors with the same length.

Step 3: Training deep learning models Vectorized code gadgets or SeVCs of source code are used as input for training deep learning models to have the ability to detect vulnerability. In this step, various DL models can be deployed to analyze provided input to recognize vulnerability. Particularly, this paper chooses 4 well-known DL models including Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Bidirectional Long Short-Term Memory (Bi-LSTM), and Transformer, which are mentioned in more detail later.

3.2.2 Detecting phase

Once DL models have been trained, Fig. 1b illustrates the steps involved in the detecting phase to recognize any vulnerability of a given piece of code.

In general, there are some similarities in both two phases, except for specific tasks for training or detecting only. For more details, just as in the training phase, the input of the detecting phase is also the source code of software that needs to be verified, as well as it also requires three steps to detect vulnerable software.

Step 1: Code representation This step also aims to extract code gadgets or SeVCs to represent the given source code with the same approach as in the training phase. However, no labeling task is done here as it is the final answer of this phase.

Step 2: Transforming to vectors The extracted representation is converted to numerical vectors by tokenization and word embedding as above-mentioned. This step also has the duty of preparing input for DL models, just as it does in the training phase.

Step 3: Vulnerability detecting In this step, the trained DL model obtained from the training phase is used to recognize any vulnerability existing in the software based on its transformed vector. The output is the decision on whether the given program is vulnerable.

3.3 Deep learning models

In the proposed framework, the DL model uses embedded data as input in the training phase to obtain vulnerability recognition capability and perform its role in the detection phase. This component can be implemented by various DL models, three of which are chosen to be mentioned below.

3.3.1 Convolutional neural network (CNN)

CNNs [26] are analogous to traditional neural networks with numerous neurons. Although they are specially designed to be primarily used to solve difficult image-driven recognition tasks, any piece of data that can be represented in the form of image vectors or pixels can be used to feed CNN models.

The general architecture of CNN used in the proposed system is depicted in Fig. 2, in which convolutional layers with ReLU as activate function and max-pooling layer are used for feature maps and full-connected layers for the classification task. The softmax is applied in the output layer to produce probabilities between 0 and 1 for two possible outputs, vulnerable or not. The time complexity of the CNN model mainly depends on convolutions, pooling, and fully connected operations. The larger input sizes and deeper architecture result in higher time complexity. For instance, a single convolutional layer has complexity as $O(k \cdot n \cdot d^2)$ where

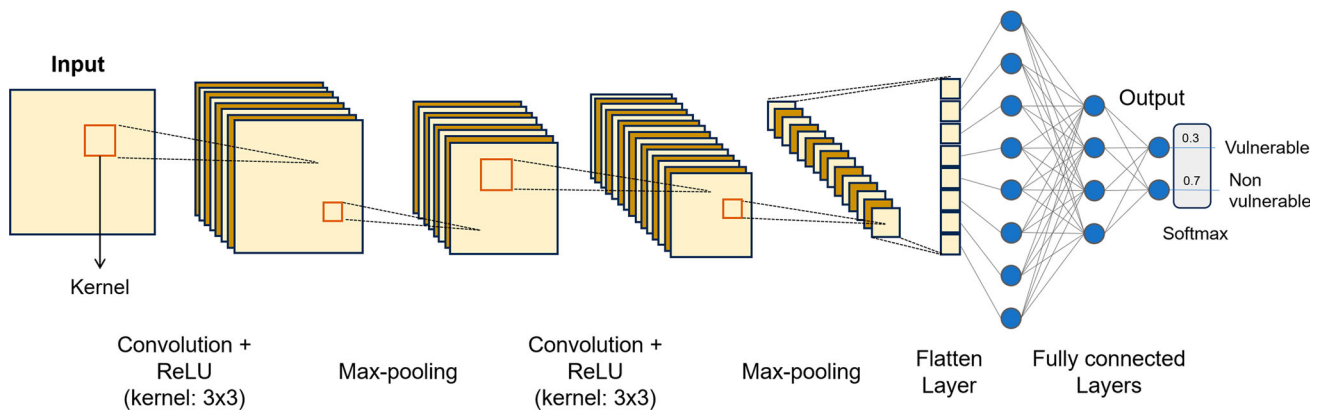
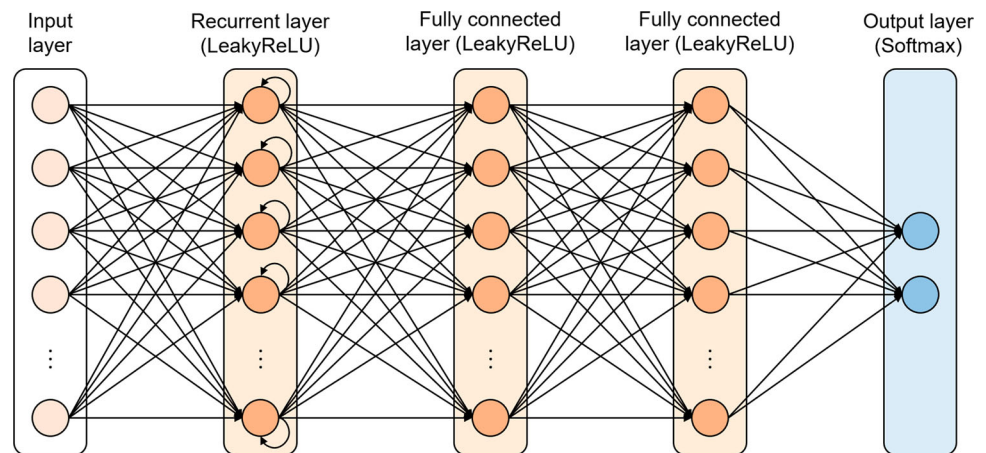


Fig. 2 Architecture of CNN model

Fig. 3 Architecture of RNN model



n, k, d is the input size, kernel size, and output dimension, respectively [33].

3.3.2 Recurrent neural network (RNN)

RNNs [24] have already been applied to a wide variety of problems involving time sequences of events and ordered data, such as characters in words. It is designed as a neural network with feedback (closed loop) connections, which represents its recurrence nature.

Figure 3 describes the underlying architecture of the RNN model in the suggested system. It is made up of a recurrent layer and 2 other fully connected layers, which are all activated by the LeakyReLU function. The softmax function is also used in the output layer. Given n as the input size and d as the output dimension, the time complexity of each recurrent layer in RNN is $O(n \cdot d^2)$ [33], which is k times less expensive than that of a convolutional layer.

3.3.3 Bidirectional long short-term memory (BiLSTM)

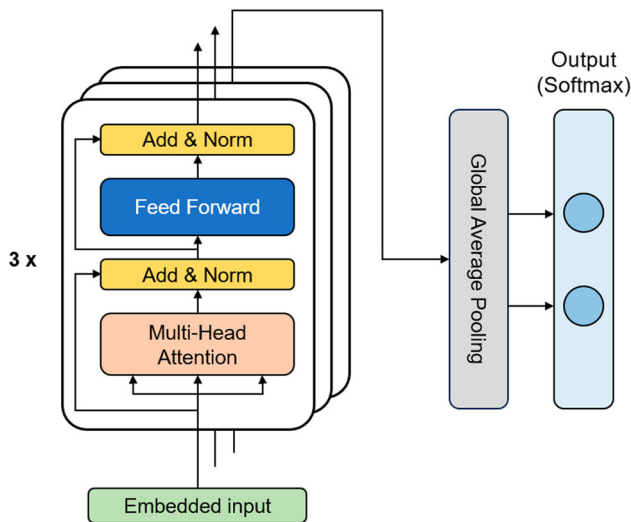
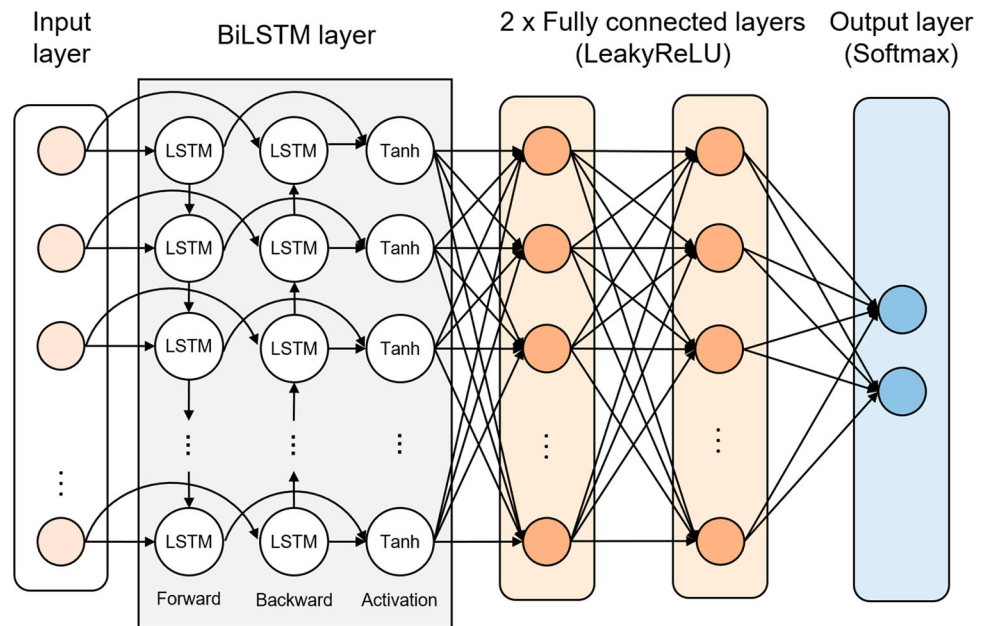
BiLSTM [27] is another variation of LSTM (upgraded RNN) with the capability of processing information in both direc-

tions forward and backward thanks to 2 LSTM layers. This allows the models to learn useful information from both directions of sequence data to capture the relationship of features.

As depicted in Fig. 4, the architecture of the BiLSTM model is quite similar to RNN, except for the BiLSTM layer with Tanh as an activation function instead of the Recurrent layer. The time complexity of each layer is considered as $O(2 \cdot n \cdot d^2)$ with the input length of n and output dimension of d due to its bidirectional nature compared to recurrent layers.

3.3.4 Transformer

Transformer [33], which relies on an attention mechanism to draw global dependencies between input and output, is an advancement in the tasks of processing natural language, computer vision, and audio processing, especially in the problem of machine translation. A typical transformer model consists of encoders and decoders. However, based on the context where the Transformer is applied, its architecture can be used in 3 different ways [21]: full architecture for sequence-to-sequence modeling such as machine translation,

Fig. 4 Architecture of BiLSTM model**Fig. 5** Architecture of the transformer model

encoder-only for text classification or sequence labeling, or decoder-only for sequence generation.

As the scope of this paper is classifying vulnerable and not vulnerable pieces of software, the Transformer model in Defect-Scanner only consists of the encoder part. This part is designed with 3 identical transformer blocks as in Fig. 5. The final output of the transformer is produced by a softmax layer, where each unit of the layer corresponds to a class of being vulnerable or not. In terms of time complexity, the Transformer is more complicated compared to RNN-based models, with the value of $O(n^2 \cdot d)$ for the self-attention layer relating to a sequence length of n [33].

4 Implementation and experiments

This section introduces the preparation of experiments, including the implementation of the Defect-Scanner framework with various NLP techniques and DL models. Thereafter, scenarios and metrics along with experimental results are mentioned.

4.1 Experimental setup

The proposed Defect-Scanner system is implemented on the Google Colab virtual machine which has a configuration of 12 GB RAM, 16 GB GPU RAM, and 80 GB hard disk and Python 3.10.12 installed. All DL models mentioned in Sect. 3.3 are trained in 30 epochs using the same hyperparameters with a *batch_size* of 16, the *learning rate* of 0.002, *Adam* as the optimizer and *categorical_crossentropy* as the loss function. Due to the diversity in values of those parameters, the chosen values are defined as the ones resulting in the best outcomes upon several customizing times. We utilized methods such as grid search or random search, depending on the complexity and computational feasibility, to explore the hyperparameter space thoroughly.

4.2 Dataset

To implement and evaluate the proposed system, 3 different datasets, called SySeVR [18], VulDeePecker [19], and VulDeBERT [16] are used in the training and testing phases.

- *SySeVR dataset* [1] is a dataset associated with the publication called SySeVR [18]. It consists of a sufficient

Table 2 Data distribution in benchmark datasets

Dataset	Record form	Number of records	Training (80%)		Testing (20%)	
			0	1	0	1
SySeVR	SeVC	420,627	291,385	45,116	72,874	11,279
VulDeePecker	Code gadget	61,638	35,130	14,200	8783	3525
VulDeBERT	Code gadget	41,509	12,598	20,609	3150	5152

number of records in the form of SeVCs extracted using the algorithm from [18] on pieces of software gathered from NVD and SARD. In 420,627 SeVCs in this dataset, there are 56,396 vulnerable ones relating to API calls, the usage of arrays, pointers, regex, etc.

- *VulDeePecker dataset* [3] is also a dataset introduced in a publication with the same name [19], using an algorithm mentioned to extract code gadgets from software's source code to use in training and testing tasks. As a result, this dataset includes code gadgets representing 61,638 pieces of software, in which 17,735 gadgets are considered vulnerable and mainly relate to buffer (CWE-119) and resource management errors (CWE-399).
- *VulDeBERT dataset* [2] is obtained using the same algorithm as VulDeePecker to extract code gadgets relating to system calls, except for some improvements [16] to eliminate code gadgets that can result in ambiguous classification results. The purpose of this task is to minimize the false positive rate due to incorrect labeling, which negatively impacts training performance.

Note that all the above datasets have records related to each type of vulnerability stored in separate folders and are labeled as “1” or “0” to identify their vulnerable nature related to the context of such type. However, as this paper aims to identify whether a target program is vulnerable, only the binary classification is performed regardless of various available vulnerability types. Hence, in the used datasets, records, either in the form of code gadgets or SeVCs, are only considered with their label of “0” or “1” to indicate their benign or vulnerable nature, ignoring the context of a specific vulnerability. Then, each dataset is split into two different parts for the training and testing phases, with a ratio of 80:20, respectively. Table 2 summarizes the distribution of those datasets in different phases.

4.3 Evaluation metrics

Given TP is the number of correctly detected vulnerable software, FP is the number of benign software indicated as vulnerable, FN counts the number of undetected vulnerable samples, and TN reflects the number of truly detected benign software, the performance of the proposed system is evaluated via 4 following metrics.

- *Accuracy*, which is calculated as in Eq. (1), measures the ratio of the number of correctly classified software to the total number of samples.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

- *Recall* is the ratio of the number of correctly detected vulnerable software to the total number of vulnerable samples, which is defined as in Eq. (2).

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

- *Precision* reflects the ratio of the number of truly vulnerable pieces of software to the total number of samples that are classified as vulnerable, as in Eq. (3).

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

- *F1-score* is the harmonic mean of Precision and Recall, which is calculated via Eq. (4).

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (4)$$

A vulnerability-detecting system is expected to effectively distinguish vulnerable and benign pieces of software with as less false predictions as possible. In other words, higher performance is reflected via higher values of all the above metrics.

4.4 Scenarios

This study aims to answer the two following questions.

- Which embedding technique, Word2Vec or CodeBERT, outperforms the remaining one when being integrated into extracting semantics of source code in detecting vulnerable software?
- Which combination of word embedding technique and deep learning models has the potential to detect vulnerable software?

To achieve this, 2 scenarios are designed to evaluate the proposed system based on different criteria.

Table 3 Performance of various DL models with two embedding techniques on code gadget-based datasets

Approach	Accuracy	Precision	Recall	F1-score
Word2Vec + BiLSTM (VulDeePecker [19])	85.06	78.55	96.47	86.59
Word2Vec + RNN	84.95	79.13	94.93	86.31
Word2Vec + CNN	84.28	77.92	95.66	85.88
Word2Vec + Transformer	82.11	75.51	95.04	84.15
CodeBERT + BiLSTM	79.03	72.72	92.9	81.58
CodeBERT + RNN	73.5	66.6	94.29	78.06
CodeBERT + CNN	80.38	77.8	85.01	81.24
CodeBERT + Transformer	80.18	75.57	89.19	81.82

Each metric has two bold values, which are the highest ones observed in 2 different cases of combining DL model with Word2Vec and CodeBERT

1. *Scenario 1—Performance in detecting vulnerabilities based on code gadgets*: In this scenario, records in 2 datasets including VulDeePecker and VulDeBERT, which are in code gadget form, are combined to make up a total dataset used to train and test DL models with the support 2 mentioned word embedding techniques, Word2Vec and CodeBERT. In particular, the combined dataset consists of 103,147 code gadgets, of which 43,486 samples are vulnerable. Via this experiment, we expect to figure out the best combination of the word embedding technique and the DL model in detecting vulnerabilities by analyzing code gadgets. Besides, a comparison of Word2Vec and CodeBERT is also considered.

All DL models are trained in 30 epochs with a batch size of 16 regardless of the used word embedding technique.

2. *Scenario 2—Performance in detecting vulnerabilities based on SeVCs*: This scenario attempts to evaluate the performance of DL models after being trained with a SeVC-based dataset, which is SySeVR. The experiment approach as well as training parameters in this scenario are similar to the first scenario, except for the used dataset.

In the two above scenarios, the performance evaluation is performed by observing 4 metrics, including Accuracy, Precision, Recall, and F1-score.

4.5 Experimental result

4.5.1 Scenario 1

The results of evaluating various combinations of DL models with Word2Vec or CodeBERT in detecting vulnerabilities using code gadgets are given in Table 3. Meanwhile, the corresponding visualization can be seen in Fig. 6, where columns with solid and dotted borders represent Word2Vec-combined and CodeBERT-assisted DL models, respectively.

In the approach of using Word2Vec, all DL models achieve acceptable performances in analyzing code gadgets when most metrics are approximately above 85%, except for Pre-

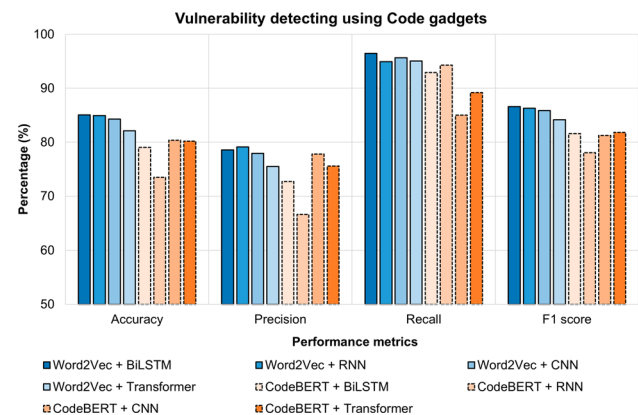


Fig. 6 Performance comparison of various combinations of word embedding and DL models in detecting vulnerabilities using code gadgets

cision. Moreover, it can be noted that the BiLSTM model, which is used in [19] to detect vulnerabilities in the VulDeePecker dataset only, also outperforms other counterparts with the highest values of Accuracy, Recall, F1-score, and the second-highest value of Precision in the combined code-gadget based dataset. Meanwhile, Transformer is the model with the worst performance when working with Word2Vec in understanding code gadgets.

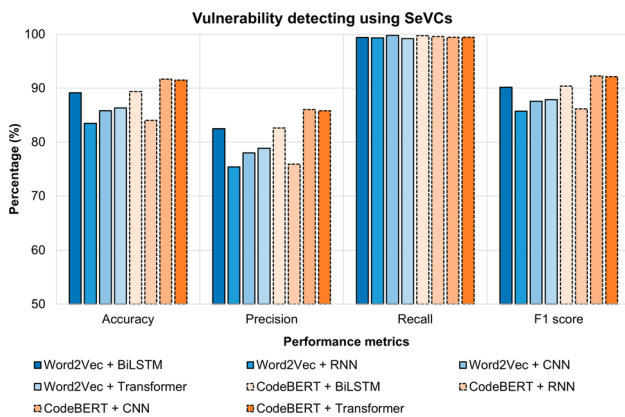
Compared to Word2Vec, DL models with the support of CodeBERT seem to be worse in terms of 4 chosen metrics with values around 80%. This can also be seen in the figure, with the group of solid-bordered columns always winning over that with dotted borders, regardless of performance metrics. Moreover, it is also difficult to figure out the best or the worst model to combine with CodeBERT in code gadget-based vulnerability recognition.

It is also worth noting that all DL models only achieve acceptable, or even low values in the Precision metric, which reflects that those models are too sensitive so that multiple benign code gadgets have been misclassified as vulnerable ones.

Table 4 Performance of various DL models with two embedding techniques on SeVC-based datasets

Approach	Accuracy	Precision	Recall	F1-score
Word2Vec + BiLSTM (SySeVR [18])	90.78	84.55	99.77	91.54
Word2Vec + RNN	83.81	75.81	99.3	85.98
Word2Vec + CNN (SySeVR [18])	87.63	80.34	99.63	88.95
Word2Vec + Transformer	87.52	80.37	99.29	88.8
CodeBERT + BiLSTM	89.37	82.6	99.76	90.37
CodeBERT + RNN	84	75.92	99.57	86.15
CodeBERT + CNN	91.67	86.03	99.43	92.25
CodeBERT + Transformer	91.49	85.8	99.44	92.12

Each metric has two bold values, which are the highest ones observed in 2 different cases of combining DL model with Word2Vec and CodeBERT

**Fig. 7** Performance comparison of various combinations of word embedding and DL models in detecting vulnerabilities using SeVCs

4.5.2 Scenario 2

Table 4 summarizes the experiment results of the testing scenario on SeVCs. A more explicit and comparable view of those results is depicted in Fig. 7.

Clearly, the Word2Vec-assisted BiLSTM model shows its effectiveness in detecting vulnerabilities from SeVCs, which is the best one among approaches using the Word2Vec technique. Moreover, RNN is least effective this time, with only 75.81% in Precision and the lowest values in other metrics, while CNN and Transformer obtain approximately comparable results. Note that using BiLSTM and CNN with Word2Vec is one of the approaches in SySeVR [18], which is mentioned to comprehensively compare previous works and other possible combinations of NLP techniques and DL models.

Moreover, it is noticeable that compared to Word2Vec, most DL models achieve better performance when dealing with SeVCs to detect vulnerabilities with the support of CodeBERT, while there is a minor decline in all performance metrics of BiLSTM. It is also worth noting that based on the extracting methods, SeVCs seem to contain richer syntax and

semantic information than code gadgets. Hence, the obtained result proves that CodeBERT emerges as a superior choice for capturing the semantic nuances of source code embedded within SeVCs. This can be the result of being trained with multiple programming languages, giving it a more profound understanding of the source code's nature. In this case, the CNN model with CodeBERT seems to be the best approach with the highest values of evaluation measures. Once again, although RNN experiences a slight increase in all metrics when combined with CodeBERT, its results are still the worst values in most metrics, proving that this model is not a suitable candidate to work with SeVCs.

5 Discussion and future works

In summary, when it comes to analyzing either code gadgets or SeVCs derived from source code, DL models, combined with the capabilities of NLP techniques, consistently deliver promising results in the field of vulnerability detection. While Word2Vec has proven to be exceptionally effective in handling code gadgets, CodeBERT emerges as a suitable choice for DL models when it comes to understanding extracted SeVCs. Moreover, among 4 deployed DL models, BiLSTM proves its effectiveness when working with the Word2Vec technique, while CNN can be considered as a potential DL model for CodeBERT-embedded data.

Despite the contribution of this paper, it is worth noting that in addition to code gadgets and SeVCs, which are the primary focus of this research, there exist other code representation methodologies, such as Abstract Syntax Trees (ASTs) and Control-flow graphs (CFGs). These alternative representations have shown potential in identifying vulnerable sections of software in some previous studies, but they have not been comprehensively evaluated in combination with various word embedding techniques, which needs further investigation.

The promising results obtained from applying natural language processing (NLP) techniques like Word2Vec and

CodeBERT to vulnerability detection in C and C++ programs suggest several avenues for future research in expanding the applicability of the Defect-Scanner framework to other programming languages and software ecosystems. One compelling direction involves adapting these NLP models to capture the syntactic and semantic nuances of different programming languages. This could be achieved by training these models on diverse programming language corpora, thus enabling them to generate contextual embeddings that reflect the unique characteristics of each language. Additionally, leveraging the flexibility of deep learning models could allow for seamless integration of these embeddings, facilitating the detection of vulnerabilities across various codebases. Moreover, implementing transfer learning strategies could prove instrumental in refining the models by utilizing the foundational knowledge gained from C and C++ to expedite and enhance learning in other programming contexts. This approach would streamline the adaptation process and improve the efficiency of the model and effectiveness in identifying vulnerabilities in new and varied software ecosystems. Besides, instead of binary classification, the capability of categorizing vulnerabilities is also a promising function to equip the framework to provide more meaningful information about possible weaknesses or further recommendations to strengthen the programs. These future directions hold the potential to significantly broaden the scope of NLP-based vulnerability detection, making it a robust tool for ensuring software security in an increasingly diverse technological landscape.

While the experimental setup for Defect-Scanner demonstrates its efficacy in identifying vulnerabilities in C and C++ programs using advanced NLP and deep learning techniques, a more detailed exploration of its practical application within real-world software development environments and its scalability to large codebases could significantly enhance the utility of the findings. Real-world implementation would necessitate an assessment of the framework's integration with existing development workflows, including continuous integration/continuous deployment (CI/CD) pipelines, and its ability to perform in dynamic, large-scale development settings. Addressing scalability, it is crucial to evaluate how well Defect-Scanner manages extensive and complex codebases that are frequently found in enterprise-level applications. This involves not only the raw processing power required but also the efficiency of its algorithms to maintain accuracy and speed in real-time analysis scenarios.

Additionally, integrating the Defect-Scanner framework to automatically scan source code commits for vulnerabilities offers a proactive approach to security. By embedding this system into version control platforms, such as Git, it can be utilized to review new commits before they are merged, catching potential security issues at the earliest possible stage. This can be tightly integrated with CI systems

to automate the security review of each commit or pull request, supporting a shift-left approach in cybersecurity that emphasizes early detection in the development lifecycle. Such real-time feedback not only prevents security flaws from advancing to production but also educates developers on secure coding practices, gradually improving code quality and security mindfulness within the development team. Utilizing Defect-Scanner for scanning source code commits optimizes the security review process and supports contemporary agile development methodologies, presenting a scalable and proficient approach to software vulnerability management.

The integration of DL and ML can significantly enhance software vulnerability detection systems. DL can automate feature extraction, enabling the system to identify complex patterns and anomalies that may indicate vulnerabilities. ML can then refine these detections, using supervised learning, to improve accuracy over time based on new data and feedback. This synergy allows for more robust and adaptive detection mechanisms, addressing the limitations of traditional methods that often rely on static, rule-based analysis like the malware detection and intrusion detection domain [4, 29].

Lastly, DL-based systems for detecting code vulnerabilities have demonstrated promising results by automating the identification of potential security threats in software. However, these systems are not immune to adversarial attacks, which can significantly undermine their effectiveness. Adversarial attacks involve manipulating the input data to the model in a way that causes the model to make a mistake. These manipulations are typically indiscernible to human reviewers but can deceive models into misclassifying malicious code as benign or overlooking critical vulnerabilities. One striking example of such vulnerabilities comes from the domain of malware detection [22, 28, 37], which closely parallels code vulnerability detection [8, 38, 39, 41]. Researchers have demonstrated that by making minimal and calculated changes to the malware's code, they can create adversarial samples that successfully bypass state-of-the-art malware detection systems without affecting the malicious functionality of the code. This type of attack can be extrapolated to systems designed to detect vulnerabilities in software, where adversarial inputs might mask serious security flaws from detection algorithms [8, 38, 39, 41].

These instances highlight a critical weakness in relying solely on pattern recognition without robust adversarial training. The need for models that can withstand such manipulations is clear, and methods like adversarial training, where models learn from both genuine and adversarially modified inputs, are becoming crucial. This approach, which is our future work, helps models generalize better and become less sensitive to the specific adversarial manipulations they might encounter in the real world. Enhancing the robustness of code vulnerability detection systems against these sophisticated

attacks is essential to maintaining the reliability and security of automated systems in cybersecurity.

6 Conclusion

In conclusion, the landscape of modern software systems is marked by complexity and rapid evolution, giving rise to an array of challenges that span from sophisticated cyber threats to the diverse variety of programming languages and coding styles. Effectively identifying vulnerabilities within this dynamic context demands advanced techniques capable of deciphering intricate patterns while accommodating the nuances of software security. This paper has risen to these challenges by harnessing the potential of cutting-edge natural language processing innovations, specifically Word2Vec and CodeBERT. By leveraging these techniques to convert code components and source code into contextual embedding vectors, this study has demonstrated their potential to significantly enhance the precision and accuracy of vulnerability identification. Through rigorous experimentation, which encompassed the utilization of the SySeVR, VulDeePecker, and VulDeBERT datasets, this paper has contributed to a more comprehensive and practical approach to software vulnerability detection. These experiments not only showcase the capabilities of Word2Vec and CodeBERT but also lay the foundation for the continued development of advanced tools in the pursuit of enhanced software security.

As we navigate the ever-evolving software landscape and confront increasingly intricate security challenges, the methods presented in this paper offer a promising pathway forward. By synergizing the power of natural language processing with the complexities of code analysis, the study can foster a new era of software vulnerability detection that is more resilient, accurate, and adaptable. This research underscores the critical role that advanced techniques play in safeguarding the integrity and security of software systems, ultimately contributing to the advancement of the broader field of cybersecurity.

Acknowledgements research was supported by The VNUHCM-University of Information Technology's Scientific Research Support Fund.

Declarations

Conflict of interest The authors have no Conflict of interest to declare that are relevant to the content of this article.

References

1. SySeVR dataset. <https://github.com/SySeVR/SySeVR>
2. VulDeBERT dataset. <https://github.com/SKKU-SecLab/VulDeBERT>
3. VulDeePecker dataset. <https://github.com/CGCL-codes/VulDeePecker>
4. Ait Messaad, B., Chetioui, K., Balboul, Y., Rhachi, H.: Analyzing and detecting malware using machine learning and deep learning. In: The International Conference on Artificial Intelligence and Smart Environment, pp. 518–525. Springer (2023)
5. Brauckmann, A., Goens, A., Ertel, S., Castrillon, J.: Compiler-based graph representations for deep learning models of code. In: Proceedings of the 29th International Conference on Compiler Construction (2020). <https://doi.org/10.1145/3377555.3377894>
6. Cheng, X., Zhang, G., Wang, H., Sui, Y.: Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022, pp. 519–531. Association for Computing Machinery, New York (2022)
7. Croft, R., Xie, Y., Babar, M.A.: Data preparation for software vulnerability prediction: a systematic literature review. *IEEE Trans. Softw. Eng.* **49**(3), 1044–1063 (2022)
8. Du, X., Wen, M., Wei, Z., Wang, S., Jin, H.: An extensive study on adversarial attack against pre-trained models of code. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, pp. 489–501. Association for Computing Machinery, New York (2023). <https://doi.org/10.1145/3611643.3616356>
9. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M.: CodeBERT: A Pre-Trained Model for Programming and Natural Languages. pp. 1536–1547 (2020). <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
10. Ghaffarian, S.M., Shahriari, H.R.: Software vulnerability analysis and discovery using machine-learning and data-mining techniques: a survey. *ACM Comput. Surv. (CSUR)* **50**(4), 1–36 (2017)
11. Hanif, H., Nasir, M.H.N.M., Razak, M.F.A., Firdaus, A., Anuar, N.B.: The rise of software vulnerability: taxonomy of software vulnerabilities detection and machine learning approaches. *J. Netw. Comput. Appl.* **179**, 103009 (2021)
12. Hariyanti, E., Djunaidy, A., Siahaan, D.: Information security vulnerability prediction based on business process model using machine learning approach. *Comput. Secur.* **110**, 102422 (2021)
13. Hin, D., Kan, A., Chen, H., Babar, M.A.: LineVD: statement-level vulnerability detection using graph neural networks. In: MSR '22: Proceedings of the 19th International Conference on Mining Software Repositories (2022)
14. Khan, R.A., Khan, S.U., Khan, H.U., Ilyas, M.: Systematic mapping study on security approaches in secure software engineering. *IEEE Access* **9**, 19139–19160 (2021)
15. Khurana, D., Koli, A., Khatter, K., Singh, S.: Natural language processing: state of the art, current trends and challenges. *Multimed. Tools Appl.* **82**(3), 3713–3744 (2023)
16. Kim, S., Choi, J., Ahmed, M.E., Nepal, S., Kim, H.: VulDeBERT: a vulnerability detection system using BERT. In: 2022 IEEE ISSREW, pp. 69–74 (2022). <https://doi.org/10.1109/ISSREW55968.2022.00042>
17. Li, J., He, P., Zhu, J., Lyu, M.R.: Software defect prediction via convolutional neural network. In: 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS) (2017)
18. Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: SySeVR: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secure Comput.* **19**(4), 2244–2258 (2022). <https://doi.org/10.1109/TDSC.2021.3051525>
19. Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: VulDeePecker: a deep learning-based system for vulnerability detection. In: NDSS Symposium (2018)

20. Lin, G., Wen, S., Han, Q.L., Zhang, J., Xiang, Y.: Software vulnerability detection using deep neural networks: a survey. *Proc. IEEE* **108**(10), 1825–1848 (2020)
21. Lin, T., Wang, Y., Liu, X., Qiu, X.: A survey of transformers. *AI Open* **3**, 111–132 (2022). <https://doi.org/10.1016/j.aiopen.2022.10.001>
22. Ling, X., Wu, L., Zhang, J., Qu, Z., Deng, W., Chen, X., Qian, Y., Wu, C., Ji, S., Luo, T., et al.: Adversarial attacks against Windows PE malware detection: a survey of the state-of-the-art. *Comput. Secur.* **128**, 103134 (2023)
23. Marjanov, T., Pashchenko, I., Massacci, F.: Machine learning for source code vulnerability detection: what works and what isn't there yet. *IEEE Secur. Priv.* **20**(5), 60–76 (2022)
24. Medsker, L., Jain, L.C.: Recurrent neural network: design and applications (2001)
25. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. In: *International Conference on Learning Representations* (2013)
26. O'Shea, K., Nash, R.: An Introduction to Convolutional Neural Networks (2015)
27. Schuster, M., Paliwal, K.: Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* **45**, 2673–2681 (1997). <https://doi.org/10.1109/78.650093>
28. Shaukat, K., Luo, S., Varadharajan, V.: A novel method for improving the robustness of deep learning-based malware detectors against adversarial attacks. *Eng. Appl. Artif. Intell.* **116**, 105461 (2022)
29. Shaukat, K., Luo, S., Varadharajan, V.: A novel deep learning-based approach for malware detection. *Eng. Appl. Artif. Intell.* **122**, 106030 (2023)
30. Shaukat, K., Luo, S., Varadharajan, V., Hameed, I.A., Chen, S., Liu, D., Li, J.: Performance comparison and current challenges of using machine learning techniques in cybersecurity. *Energies* **13**(10), 2509 (2020)
31. Shaukat, K., Luo, S., Varadharajan, V., Hameed, I.A., Xu, M.: A survey on machine learning techniques for cyber security in the last decade. *IEEE Access* **8**, 222310–222354 (2020)
32. Tang, W., Tang, M., Ban, M., Zhao, Z., Feng, M.: CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection. *J. Syst. Softw.* (2023)
33. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems* (2017). [arXiv:1706.03762pdf](https://arxiv.org/abs/1706.03762)
34. Viet Phan, A., Le Nguyen, M., Thu Bui, L.: Convolutional neural networks over control flow graphs for software defect prediction. In: *2017 IEEE 29th International Conference on Tools with Artificial Intelligence (ICTAI)* (2017). <https://doi.org/10.1109/ICTAI.2017.00019>
35. Wang, S., Liu, T., Tan, L.: Automatically learning semantic features for defect prediction. In: *ICSE '16: Proceedings of the 38th International Conference on Software Engineering* (2016)
36. Wu, J.: Literature review on vulnerability detection using NLP technology. [arXiv:2104.11230](https://arxiv.org/abs/2104.11230) (2021)
37. Yan, S., Ren, J., Wang, W., Sun, L., Zhang, W., Yu, Q.: A survey of adversarial attack and defense methods for malware classification in cyber security. *IEEE Commun. Surv. Tutor.* **25**(1), 467–496 (2022)
38. Yang, Y., Fan, H., Lin, C., Li, Q., Zhao, Z., Shen, C.: Exploiting the adversarial example vulnerability of transfer learning of source code. *IEEE Trans. Inf. Forensics Secur.* **19**, 5880–5894 (2024). <https://doi.org/10.1109/TIFS.2024.3402153>
39. Yu, X., Li, Z., Huang, X., Zhao, S.: Advulcode: Generating adversarial vulnerable code against deep learning-based vulnerability detectors. *Electronics* **12**(4), 936 (2023)
40. Zeng, P., Lin, G., Pan, L., Tai, Y., Zhang, J.: Software vulnerability analysis and discovery using deep learning techniques: a survey. *IEEE Access* **8**, 197158–197172 (2020)
41. Zhang, H., Lu, S., Li, Z., Jin, Z., Ma, L., Liu, Y., Li, G.: Codebert-attack: adversarial attack against source code deep learning models via pre-trained model. *J. Softw. Evol. Process* **36**(3), e2571 (2024)
42. Zhang, Q., Wu, B.: Software defect prediction via transformer. In: *2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)* (2020). <https://doi.org/10.1109/ITNEC48623.2020.9084745>
43. Zhu, Y., Lin, G., Song, L., Zhang, J.: The application of neural network for software vulnerability detection: a review. *Neural Comput. Appl.* **35**(2), 1279–1301 (2023)
44. Ziem, N., Wu, S.: Security vulnerability detection using deep learning natural language processing. In: *IEEE INFOCOM 2021* (2021). <https://doi.org/10.1109/INFOCOMWKSHPS51825.2021.9484500>
45. Zou, D., Wang, S., Xu, S., Li, Z., Jin, H.: VulDeePecker: a deep learning-based system for multiclass vulnerability detection. *IEEE Trans. Dependable Secure Comput.* **18**(5), 2224–2236 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.